

App Building Platforms for Commission Agents

https://gemini.google.com/app/1d85b8284f99555a

User prompt: to build an app, which one do you recommend for commission agent

Response: To recommend the right platform for a commission agent app, the choice depends heavily on **who uses the app** and **what specific workflow it handles**. Commission agent apps typically require handling ledger tracking, client management, real-time commission calculations, receipt generation, and multi-role access (agent vs. manager/broker).

Here are the top recommendations categorized by your build approach:

1. Top Pick for Fast No-Code/Low-Code Apps: Bubble or FlutterFlow

If you want to build visually without writing code line-by-line, these are the leading platforms:

- FlutterFlow (Best for Native Mobile Apps - iOS & Android):
 - Why it fits:** Perfect if your agents need an app on their phones in the field. It handles offline data sync, mobile notifications, PDF generation (for receipts/invoices), and multi-currency calculations natively.
 - Tech Stack:** Exports clean Flutter/Dart code if you ever decide to take ownership of the codebase.
- Bubble (Best for Web Apps & Complex Logic):
 - Why it fits:** Handles deep backend business logic out of the box (e.g., tiered commission rules, automatic payout splits, user permission management for agents vs. agency owners). Excellent for web dashboards.

2. Top Pick for Internal Tools & Quick Database Apps: Softr or Glide

If your app is mainly for internal record-keeping, client tracking, and commission log management:

- Softr + Airtable / Google Sheets:
 - Why it fits:** Extremely fast to set up. You can map your clients, sales transactions, and commission tiers in Airtable or Google Sheets, and Softr automatically generates a clean user interface with agent sign-in portals.
- Glide:
 - Why it fits:** Turns Google Sheets or Excel files into a mobile-friendly web app in hours. Ideal for solo agents or small teams tracking sales on the go.

3. Top Pick for Full-Stack Custom Code (Dev-Led): React / Next.js + Supabase

If you are a developer (or hiring one) and want total control, offline functionality, and no recurring platform limits:

- Frontend:** React / Next.js (Web) or React Native (Mobile)
- Backend & Database:** Supabase (PostgreSQL) or Firebase
- Why it fits:** Complete flexibility to handle complex ledger equations, automated SMS/Telegram notifications for commission alerts, and offline transaction logging (using IndexedDB or local storage) for low-connectivity environments.

Comparison Matrix

Platform	Best For	Deployment	Time to Build	Learning Curve
Glide / Softr	Internal tracking, fast MVPs	Web / Progressive Web App	1–3 days	Low
Bubble	Custom web portals, complex roles	Web / Web-Mobile	1–3 weeks	Medium
FlutterFlow	Native mobile app (iOS/Android)	App Store / Google Play	2–4 weeks	Medium-High
React + Supabase	Scalable commercial SaaS / Enterprise	Web / Native Mobile	4+ weeks	High (Code)

Core Features Your App Should Include

Regardless of the platform you choose, ensure your database schema covers:

- Transaction & Sales Ledger:** Sales value, product/service type, date, and status (Pending/Approved/Paid).
- Commission Engine:** Base percentage, fixed-rate rules, or tiered rates (e.g., 5% up to \$10k, 8% above \$10k).
- Multi-Role Access Control:** Separate views for field agents (see only their sales) and agency admin (approve commissions, export summaries).
- Document/Receipt Export:** Generating PDF payout vouchers or sales summaries for clients and accounting.

To help narrow down the exact platform, **what industry is the commission agent operating in (e.g., real estate, stationers/supplies, brokerage, retail), and will this be used primarily on mobile phones or web dashboards?**

Sources

- [The 5 best AI app builders in 2026 - Zapier](#)

User prompt: mobile users and consider social media app builder apps too

Response: When you introduce **mobile-first deployment** and **social media features** (like chat feeds, media uploads, real-time messaging, and profile networking) into a commission agent app, the tech choices shift.

A social media feature set requires handling high volumes of multimedia, push notifications, real-time updates, and native device performance. Here are the top platforms to build a mobile-first app with built-in or easy-to-integrate social features:

1. Top Pick Overall: FlutterFlow + Firebase

If you want a true mobile app (iOS & Android) with a social feed, direct chat, and commission logs, **FlutterFlow** is the absolute best fit.

- Why it works for Social Media Features:**

- **Real-time Chat & Feeds:** Integrates natively with **Firebase**, Google’s real-time database. This makes live updates (like instant messaging between agents, live activity feeds, or real-time commission notifications) smooth and responsive.
 - **Media Uploads:** Native support for camera access, video playback, audio messaging, and photo uploads for sales receipts or property showcase feeds.
 - **Push Notifications:** Native integration with Firebase Cloud Messaging (FCM) to alert agents on comment replies, new messages, or commission payouts.
- **Build Effort:** 2–4 weeks.

2. Dedicated Community & Social App Builders: Mighty Networks or Circle

If your app focuses heavily on creating an exclusive agent network, community feeds, discussion forums, and direct messaging (with commission tracking kept relatively simple), specialized community platforms save massive development time.

- **Mighty Networks / Circle:**
 - **Why it fits:** Out-of-the-box social feeds, user profiles, group chats, direct messaging, events, and member directories.
 - **Best Used For:** Agent networks, training, community discussions, and referral boards where formal ledger logic is secondary or linked via external forms.

3. Fast Mobile Prototyping: Adalo or GoodBarber

If FlutterFlow has too steep of a learning curve and you want drag-and-drop mobile social features quickly:

- **Adalo:**
 - **Why it fits:** Very easy to drag and drop social components like user profiles, like/comment buttons, social activity feeds, and simple messaging apps.
 - **Trade-off:** Performance can struggle if the app grows to tens of thousands of real-time transactions or heavy media uploads.
- **GoodBarber:**
 - **Why it fits:** Strong out-of-the-box native mobile UI for news feeds, social networking, media submission, and push notifications.

4. Native Custom Code: React Native (Expo) + Supabase

If you have a developer team building a custom, highly scalable platform with custom chat protocols, media streaming, and complex multi-tier commissions:

- **React Native (Expo):** React Native handles native mobile UI smoothly, and **Supabase Realtime** gives you instant messaging, live commission logs, and file storage without managing raw servers.

Platform Selection Guide

Platform	Native Mobile Output?	Social Features (Feeds/Chat)	Ledger & Logic Complexity	Recommendation
FlutterFlow	✅ iOS & Android	High (via Firebase)	High	#1 Choice for Custom App
Mighty Networks	✅ iOS & Android	Built-in	Low (Community focus)	Best for Agent Networks
Adalo	✅ iOS & Android	High (Drag & drop)	Medium	Best for Quick MVP
React Native	✅ iOS & Android	Unlimited (Custom)	Unlimited	Best for Full Developer Control

Key Social & Mobile Features to Include

- **Activity Feed:** Showcase successful sales, top-performing agents, or newly listed commissionable inventory/leads.
- **Direct Messaging & Groups:** Private chat between agents and managers to approve sales or verify receipts before commission payout.
- **Push Notifications:** Instant mobile alerts when a transaction status updates from *Pending* to *Approved*.
- **Media Uploads:** Easy camera attachment for receipt scanning, client signatures, and product photos.

If you are looking to see how visual app builders compare on mobile capabilities, [FlutterFlow vs Bubble comparison](#) gives a clear breakdown of native mobile performance versus web platforms.

Sources

1. Top Social Media App Development Companies in 2026 - Medium
2. FlutterFlow vs Bubble: Best No-Code App Builder - Elementor
3. Best App Builders for Digital Creators in 2026 | Breakthrough
4. Glide vs FlutterFlow: Which No-Code App Builder Should You Use?
5. 7 Best Mobile App Development Platforms to Choose in 2026

User prompt: what about telegram app builder

Response: Building inside Telegram is a great route for a **commission agent app**, particularly in markets with high Telegram adoption.

Instead of traditional mobile apps, Telegram uses **Telegram Mini Apps (TMAs)**—web apps rendered natively inside a chat window—and **Telegram Bots**. This model eliminates download friction, leverages built-in user identity (phone verification), and provides immediate push notifications.

Here are the top tools and approaches to build a Telegram-based commission app, broken down by build strategy:

1. Low-Code / Visual Builders for Telegram

If you want a visual builder to construct a Mini App without heavy custom code:

- **MaviBot or Bothelp (For Business Mini Apps & Stores):**
 - **How it works:** Provides visual drag-and-drop builders designed specifically to output Telegram Web Apps (TWAs).
 - **Best for:** Agents who need simple storefronts, service booking, lead forms, and digital payment collection directly inside Telegram.
- **FlowX / ManyChat + Web App Link:**
 - **How it works:** Handles the conversational flow (lead routing, text updates, automated messaging), with custom web forms linked via Telegram's native WebApp button.

- **Glide / Softr (Embedded inside Telegram):**
 - **How it works:** Build a web portal in Glide or Softr using Google Sheets or Airtable as the database. Attach the published URL to a Telegram Bot using telegram.org/js/telegram-web-app.js. When agents open the bot, your web app loads instantly inside Telegram.

2. AI Prompt-to-App Builders (Fastest Prototyping)

If you want a custom Mini App built instantly via natural language prompts:

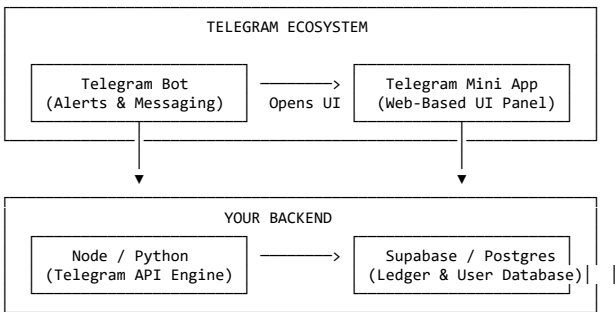
- **Blink.new / v0 / Bolt.new:**
 - **How it works:** Prompt an AI generator to build a web dashboard (React, Vite, or Next.js) tailored for commission agents. Include the Telegram WebApp SDK script in the code.
 - **Result:** Generates an interface with direct database storage (Supabase/Firebase) that fits into Telegram's bottom-sheet mobile UI.

3. Full Custom Developer Stack (Best for Production & Scalability)

For serious multi-tier ledger tracking, offline caching, and security:

- **Frontend:** React / Vite / Next.js or Vue.js (using @telegram-apps/sdk or @twa-dev/sdk).
- **Backend:** Python (python-telegram-bot / aiogram) or Node.js (grammy / telegraf).
- **Database:** Supabase (PostgreSQL) or MongoDB Atlas.
- **Why this approach wins:** Complete control over ledger formulas, custom user role management (Agents vs. Managers), PDF receipt generation, and real-time chat hooks.

How the Telegram Commission App Architecture Works



1. **User Auth:** Agents tap the bot. Telegram passes their verified Telegram user_id and username directly to your app—no sign-up passwords required.
2. **Sales Logging & UI:** Tapping "Open Dashboard" slides up a Web UI (Mini App) where agents record new transactions, upload receipt images, or track payouts.
3. **Bot Notifications:** When a manager approves a sale, the backend triggers the Telegram Bot to send a direct message to the agent: "Sale #1024 Approved! Commission: \$150."
4. **Community/Social Group:** The app can be tied to a Telegram Channel or Group where top agents appear on a public leaderboard or post deal updates.

Key Considerations for Telegram

- **Strengths:**
 - No App Store submission delays or approvals.
 - Instant access for users who already have Telegram installed.
 - Free push notifications via bot messages.
- **Weaknesses:**
 - Hardware access (e.g., advanced camera features, background GPS tracking) is more limited compared to a fully native iOS/Android app.
 - Relies entirely on the Telegram ecosystem (users must have Telegram accounts).

If you are open to developer-led or low-code options, **React/Vite with Supabase hosted inside a Telegram Mini App** offers the best balance of speed, zero app-store friction, and complete backend control.

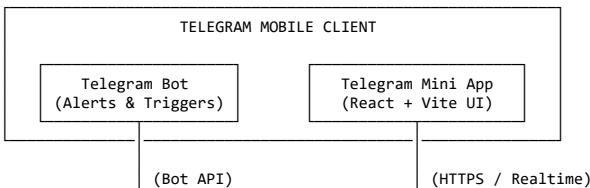
Sources

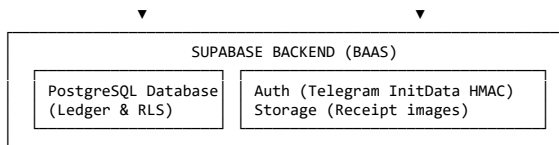
1. [Telegram Mini Apps for Business: Build One Without Code](#)
2. [Telegram Bots & Mini Apps | CimpleO Group](#)
3. [Telegram Mini App Bot - built with AI in 1 minute.](#)

User prompt: lets build using Full Custom Developer Stack

Response: Here is the architecture blueprint and starter implementation for a production-ready Commission Agent platform using **React/Vite + Supabase** embedded inside a **Telegram Mini App (TMA)**.

Architecture Overview





Database Schema (Supabase SQL)

Run this script in your Supabase SQL Editor to establish roles, transaction logs, commission calculations, and Row Level Security (RLS).

```
-- Enable UUID extension
create extension if not exists "uuid-ossf";

-- 1. USERS & ROLES
create type user_role as enum ('agent', 'manager', 'admin');

create table public.profiles (
  id uuid references auth.users on delete cascade primary key,
  telegram_id bigint unique not null,
  full_name text not null,
  role user_role default 'agent',
  created_at timestamp with time zone default timezone('utc'::text, now()) not null
);

-- 2. TRANSACTIONS & COMMISSIONS
create type transaction_status as enum ('pending', 'approved', 'rejected', 'paid');

create table public.transactions (
  id uuid default uuid_generate_v4() primary key,
  agent_id uuid references public.profiles(id) not null,
  client_name text not null,
  sale_amount numeric(12, 2) not null check (sale_amount > 0),
  commission_rate numeric(5, 2) default 5.00, -- e.g., 5.00%
  commission_amount numeric(12, 2) generated always as (sale_amount * (commission_rate / 100)) stored,
  status transaction_status default 'pending',
  receipt_url text,
  notes text,
  created_at timestamp with time zone default timezone('utc'::text, now()) not null
);

-- 3. ROW LEVEL SECURITY (RLS)
alter table public.profiles enable row level security;
alter table public.transactions enable row level security;

-- Agents can only read/write their own transactions
create policy "Agents view own transactions" on public.transactions
  for select using (auth.uid() = agent_id);

create policy "Agents insert own transactions" on public.transactions
  for insert with check (auth.uid() = agent_id);

-- Managers can view and update all transactions
create policy "Managers view all transactions" on public.transactions
  for select using (
    exists (select 1 from public.profiles where id = auth.uid() and role in ('manager', 'admin'))
  );

create policy "Managers update transactions" on public.transactions
  for update using (
    exists (select 1 from public.profiles where id = auth.uid() and role in ('manager', 'admin'))
  );
```

Frontend Code Blueprint (React + Vite)

1. Bridge Setup (src/telegram.ts)

Initialize the Telegram SDK to sync with the native frame and auto-adapt theme colors:

```
import { init, backButton, mainButton, miniApp } from '@telegram-apps/sdk';

export function setupTelegramBridge() {
  init();
  miniApp.mount();
  miniApp.setHeaderColor('secondary_bg_color');

  // Sync native viewport and ready signal
  miniApp.ready();
}
```

2. Telegram Auth Helper (src/services/auth.ts)

Authenticate users using Telegram's cryptographically signed initData:

```
import { supabase } from './supabaseClient';
import { retrieveLaunchParams } from '@telegram-apps/sdk';

export async function authenticateTelegramUser() {
  const { initDataRaw, initData } = retrieveLaunchParams();

  if (!initDataRaw || !initData?.user) {
    throw new Error('Not running inside Telegram context.');
```

```

const { data, error } = await supabase.auth.signInWithPassword({
  email: `${initData.user.id}@telegram.user`,
  password: initDataRaw, // Verified server-side via Supabase Edge Function
});

if (error) console.error('Auth error:', error.message);
return data;
}

```

3. Agent Dashboard (src/components/AgentDashboard.tsx)

A mobile UI displaying ledger totals and real-time transaction updates:

```

import React, { useEffect, useState } from 'react';
import { supabase } from '../services/supabaseClient';

interface Transaction {
  id: string;
  client_name: string;
  sale_amount: number;
  commission_amount: number;
  status: 'pending' | 'approved' | 'rejected' | 'paid';
}

export const AgentDashboard: React.FC<{ agentId: string }> = ({ agentId }) => {
  const [transactions, setTransactions] = useState<Transaction[]>([]);

  useEffect(() => {
    // Initial Fetch
    const fetchTransactions = async () => {
      const { data } = await supabase
        .from('transactions')
        .select('*')
        .eq('agent_id', agentId)
        .order('created_at', { ascending: false });
      if (data) setTransactions(data);
    };

    fetchTransactions();

    -- Live Supabase Subscription for real-time payout updates
    const subscription = supabase
      .channel('agent_ledger')
      .on(
        'postgres_changes',
        { event: '*', schema: 'public', table: 'transactions', filter: `agent_id=eq.${agentId}` },
        (payload) => {
          fetchTransactions();
        }
      )
      .subscribe();

    return () => {
      supabase.removeChannel(subscription);
    };
  }, [agentId]);

  const totalEarned = transactions
    .filter((t) => t.status === 'approved' || t.status === 'paid')
    .reduce((sum, t) => sum + Number(t.commission_amount), 0);

  return (
    <div style={{ padding: '16px', color: 'var(--tg-theme-text-color)' }}>
      <div style={{ background: 'var(--tg-theme-secondary-bg-color)', padding: '16px', borderRadius: '12px' }}>
        <h3>Total Earned Commission</h3>
        <h2>${totalEarned.toFixed(2)}</h2>
      </div>

      <h3>Recent Sales</h3>
      <div style={{ display: 'flex', flexDirection: 'column', gap: '8px' }}>
        {transactions.map((t) => (
          <div key={t.id} style={{ padding: '12px', borderBottom: '1px solid #333' }}>
            <div style={{ display: 'flex', justifyContent: 'space-between' }}>
              <strong>{t.client_name}</strong>
              <span>${t.commission_amount}</span>
            </div>
            <small style={{ color: t.status === 'approved' ? '#4CAF50' : '#FF9800' }}>
              Status: {t.status.toUpperCase()}
            </small>
          </div>
        ))}
      </div>
    </div>
  );
};

```

Telegram Bot Handler (bot.py)

A backend bot script using python-telegram-bot to launch the app and send instant commission alerts:

```

import os
from telegram import Update, InlineKeyboardButton, InlineKeyboardMarkup, WebAppInfo
from telegram.ext import ApplicationBuilder, CommandHandler, ContextTypes

TELEGRAM_BOT_TOKEN = os.getenv("TELEGRAM_BOT_TOKEN")
MINI_APP_URL = "https://your-mini-app.vercel.app"

async def start(update: Update, context: ContextTypes.DEFAULT_TYPE):

```

```
keyboard = [
    [InlineKeyboardButton("Open Commission Dashboard", web_app=WebAppInfo(url=MINI_APP_URL))]
]
reply_markup = InlineKeyboardMarkup(keyboard)
await update.message.reply_text(
    f"Welcome {update.effective_user.first_name}! Log sales and track your commissions below:",
    reply_markup=reply_markup
)

if __name__ == "__main__":
    app = ApplicationBuilder().token(TELEGRAM_BOT_TOKEN).build()
    app.add_handler(CommandHandler("start", start))
    app.run_polling()
```

Deployment & Registration Steps

- 1

Deploy Web App

Vite + Supabase Hosting

Deploy your frontend build to Vercel, Netlify, or Cloudflare Pages to obtain an SSL-enabled HTTPS URL (required by Telegram).
- 2

Register App with BotFather

Telegram Config

Open Telegram, message **@BotFather**, select your bot, and navigate to Bot Settings > Menu Button > Configure menu button. Paste your hosted app HTTPS URL.
- 3

Configure Database Sync

Supabase Webhooks

Create a Database Webhook in Supabase on public.transactions updates. Direct the payload to an HTTP endpoint on your python bot server to trigger direct Telegram messages when sale statuses change.

Sources

1. [A real-time-ready boilerplate with Vite, React and Supabase](#)

User prompt: no, before the code, as u r an expert in system analysis and design, ask me the business logic question to respond then we will proceed to the code and deployment and test

Response: Before writing a single line of code, we need to map the exact mechanics of your business operation. As system designers, defining these edge cases early will dictate our database schema, UI flows, and backend triggers.

Here are the critical business logic questions to address:

1. Commission Structure & Calculations

- **Rate Model:** Is the commission a flat percentage across all sales (e.g., standard 5%), or does it vary?
 - *If it varies:* Is it based on product categories, agent performance tiers, client types, or custom negotiated rates per deal?
- **Tiering:** Do commission percentages increase based on monthly volume thresholds (e.g., 5% on the first \$10k, 8% thereafter)?
- **Split Commissions:** Can multiple agents share a single sale or split a payout?

2. Sales Lifecycle & Multi-Role Approvals

- **Status Workflow:** What exact stages does a deal go through? (e.g., *Draft* → *Submitted* → *Under Review* → *Approved* → *Paid*)
- **Approval Authority:** Who verifies and approves a reported sale? Are there direct managers, store owners, or accounting staff who handle payouts?
- **Verification Proof:** Do agents need to upload media/documents (e.g., bank deposit receipts, signed client agreements, cash photos) when logging a sale?

3. Financials, Currencies & Ledger Rules

- **Multi-Currency:** Will transactions occur in multiple currencies (e.g., ETB vs. USD), and do rates need automatic conversion?
- **Payout Triggers:** Are agents paid immediately per sale, or batched (weekly/monthly payouts)?
- **Adjustments & Refunds:** What happens if a client cancels or refunds a purchase after a commission was logged or paid out? Is there a chargeback system?

4. Agent Operations & Social/Team Dynamics

- **Lead/Inventory Ownership:** Do agents log sales against a shared central catalog/inventory, or do they enter client names and figures freely?
- **Visibility Boundaries:** Should agents only see their own earnings, or should there be shared team leaderboards, public group announcements, or group feeds when someone closes a major deal?
- **Agent Onboarding:** How are new agents onboarded—via open self-registration, or strictly by manager invitation/approval?

Which of these rules apply to your current operation, and are there any unique workflows specific to your business model?

User prompt: that is what i mean, i just want to build an app that accepts property most condo information like block no, floor no, house no, pictures of the property, phone no of the user who register the property and store the data, broadcast to all users of the group or channel or anyone who joins the group or channel, update status of the property by the user who register it as sold, rented or occupied for sales, rental and guest house accommodations respectively, application owner will levy some money after system works out later,user who register it may set commission percentile while registering

Response: Mapping out a **condo real estate listing & broadcast bot inside Telegram** requires addressing several core operational scenarios.

Here are the key business logic questions to settle before defining the database schema and system architecture:

1. Listing Structure & Location Data

- **Condo Site Identification:** Besides **Block No**, **Floor No**, and **House No**, do users need to select the specific Condo Site/Neighborhood (e.g., *Ayat 49, Gotera, Summit, Gafa, Jeka, Bole Bulbula*)?

- **Property Details:** Should listings include specific bedroom count (e.g., Studio, 1-Bed, 2-Bed, 3-Bed), bathroom count, or pricing figures (e.g., total price or monthly rent) alongside the commission rate?

2. Broadcast & Notification Logic

- **Target Channels:** Will listings auto-post to a public Telegram Channel/Group immediately upon submission, or do they require admin review first?
- **Contact Privacy:** Should the registrant's phone number be publicly visible in the broadcasted channel message, or hidden behind a Telegram button (e.g., "Click to Contact Agent/Owner")?
- **Status Sync:** When a property status updates to **Sold**, **Rented**, or **Occupied**, should the original Telegram broadcast post automatically update its text (e.g., adding 🟡 [STATUS: RENTED]), or should a new notification be sent to the group?

3. Commission & Revenue Mechanics

- **Commission Model:** When a user sets a commission percentage (e.g., 2% for sales, 100% of first month for rentals), is this total commission earned on the deal, or the portion split with your application?
- **Platform Fee Levy:** How and when will the application collect its levy?
 - Will it be a flat fee per listing, a percentage of the earned commission, or a subscription fee?
 - How will the system verify that a transaction actually completed outside the app before levying the fee?

4. User Roles & Editing Access

- **Registrant Management:** Can brokers/agents manage multiple property listings under one account, or is each phone number treated as an independent user?
- **Listing Expiry:** Do listings remain active indefinitely, or automatically expire/archive if not updated within a set period (e.g., 30 days)?

How would you like to handle these workflows for your platform?

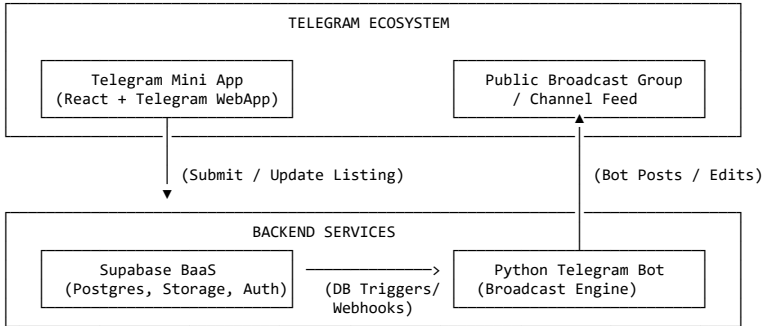
Sources

1. Condo for Rent in Addis Ababa - Jiji Ethiopia

User prompt: Listing Structure & Location Data Condo Site Identification: Besides Block No, Floor No, and House No, do users need to select the specific Condo Site/Neighborhood (e.g., Ayat 49, Gotera, Summit, Gafa, Jeka, Bole Bulbula)? yes Property Details: Should listings include specific bedroom count (e.g., Studio, 1-Bed, 2-Bed, 3-Bed), bathroom count, or pricing figures (e.g., total price or monthly rent) alongside the commission rate? yes 2. Broadcast & Notification Logic Target Channels: Will listings auto-post to a public Telegram Channel/Group immediately upon submission, or do they require admin review first? No admin review Contact Privacy: Should the registrant's phone number be publicly visible in the broadcasted channel message, or hidden behind a Telegram button (e.g., "Click to Contact Agent/Owner")? On click to contact agent/owner Status Sync: When a property status updates to Sold, Rented, or Occupied, should the original Telegram broadcast post automatically update its text (e.g., adding 🟡 [STATUS: RENTED]), or should a new notification be sent to the group? yes 3. Commission & Revenue Mechanics Commission Model: When a user sets a commission percentage (e.g., 2% for sales, 100% of first month for rentals), is this total commission earned on the deal, or the portion split with your application? On the deal not shared with the application for the time being for the app to be used many users then after reaching some numbers the app will incur users for registration Platform Fee Levy: How and when will the application collect its levy? Will it be a flat fee per listing, a percentage of the earned commission, or a subscription fee? no How will the system verify that a transaction actually completed outside the app before levying the fee? Not for the time being 4. User Roles & Editing Access Registrant Management: Can brokers/agents manage multiple property listings under one account, or is each phone number treated as an independent user? Can multiple property listing Listing Expiry: Do listings remain active indefinitely, or automatically expire/archive if not updated within a set period (e.g., 30 days)? indefinitely

Response: Now that the requirements are clear, here is the complete system specification and technical implementation blueprint tailored for your Telegram Condo Listing Platform.

System Architecture & Data Flow



1. Relational Database Schema (Supabase SQL)

Run this script in your Supabase SQL Editor. It sets up users, property listings, photos, and auto-calculates total commission values.

```
create extension if not exists "uuid-oss";

-- 1. ENUMS
create type listing_category as enum ('sale', 'rental', 'guesthouse');
create type listing_status as enum ('available', 'sold', 'rented', 'occupied');
create type bedroom_type as enum ('studio', '1-bed', '2-bed', '3-bed', '4-bed+');

-- 2. USERS / BROKERS TABLE
create table public.users (
  id uuid default uuid_generate_v4() primary key,
  telegram_id bigint unique not null,
  full_name text not null,
  phone_number text not null,
  created_at timestamp with time zone default timezone('utc'::text, now()) not null
);

-- 3. CONDO LISTINGS TABLE
create table public.properties (
  id uuid default uuid_generate_v4() primary key,
  user_id uuid references public.users(id) on delete cascade not null,
  category listing_category not null,
```

```

status listing_status default 'available' not null,

-- Location Details
site_name text not null, -- e.g., 'Ayat 49', 'Gotera', 'Koye Feche'
block_no text not null,
floor_no text not null,
house_no text not null,

-- Specs & Pricing
bedrooms bedroom_type not null,
bathrooms integer default 1 not null,
asking_price numeric(14, 2) not null, -- ETB amount
commission_percent numeric(5, 2) not null, -- e.g. 2.00%

-- Media & Telegram Integration
image_urls text[] default '{}':text[],
telegram_message_id bigint, -- Tracks broadcast post to edit status dynamically

created_at timestamp with time zone default timezone('utc':text, now()) not null,
updated_at timestamp with time zone default timezone('utc':text, now()) not null
);

-- Indexing for fast search filtering
create index idx_properties_site on public.properties(site_name);
create index idx_properties_status on public.properties(status);

```

2. Telegram Broadcast Bot Engine (bot.py)

This Python bot handles two main tasks:

1. Receives payload from your WebApp to **auto-post new listings** with photos and a dynamic "Contact Owner" button.
2. **Edits the original Telegram channel message** when status changes (e.g., adding 🟢 [STATUS: RENTED]).

```

import os
import requests
from telegram import InlineKeyboardButton, InlineKeyboardMarkup, InputMediaPhoto, Update
from telegram.ext import ApplicationBuilder, CommandHandler, CallbackQueryHandler, ContextTypes

TELEGRAM_BOT_TOKEN = os.getenv("TELEGRAM_BOT_TOKEN")
TARGET_CHANNEL_ID = "@your_condo_channel_id" # Replace with Channel/Group Username

# 1. BROADCAST NEW LISTING TO CHANNEL
async def broadcast_new_listing(app, property_data, owner_phone):
    # Construct Post Text
    status_tag = "🟢 [AVAILABLE]"
    text = (
        f"{status_tag}\n"
        f"🏠 **CONDO FOR {property_data['category'].upper()}**\n\n"
        f"📍 **Location:** {property_data['site_name']} (Block {property_data['block_no']}, Floor {property_data['floor_no']})\n\n"
        f"🛏️ **Bedrooms:** {property_data['bedrooms']}\n\n"
        f"🛁 **Bathrooms:** {property_data['bathrooms']}\n\n"
        f"💰 **Price:** {property_data['asking_price']:}, ETB\n\n"
        f"💵 **Commission Rate:** {property_data['commission_percent']}%\n\n"
    )

    # Privacy Contact Button (Reveals contact on click or opens direct call link)
    keyboard = InlineKeyboardMarkup([
        [InlineKeyboardButton("📞 Contact Agent / Owner", callback_data=f"contact_{owner_phone}")]]
    ])

    images = property_data.get('image_urls', [])
    if images:
        media_group = [InputMediaPhoto(media=img) for img in images[:10]]
        media_group[0].caption = text
        media_group[0].parse_mode = 'Markdown'
        messages = await app.bot.send_media_group(chat_id=TARGET_CHANNEL_ID, media=media_group)
        # Send inline button under the media group as a separate follow-up or main post
        sent_msg = await app.bot.send_message(chat_id=TARGET_CHANNEL_ID, text="Click below to inquire:", reply_markup=keyboard)
        return sent_msg.message_id
    else:
        sent_msg = await app.bot.send_message(chat_id=TARGET_CHANNEL_ID, text=text, reply_markup=keyboard, parse_mode='Markdown')
        return sent_msg.message_id

# 2. UPDATE EXISTING BROADCAST STATUS
async def update_broadcast_status(app, message_id, new_status, property_data):
    status_emoji = "🟢" if new_status == "available" else "🔴"

    updated_text = (
        f"{status_emoji} **[STATUS: {new_status.upper()}]**\n\n"
        f"🏠 **CONDO FOR {property_data['category'].upper()}**\n\n"
        f"📍 **Location:** {property_data['site_name']} (Block {property_data['block_no']})\n\n"
        f"🛏️ **Bedrooms:** {property_data['bedrooms']}\n\n"
        f"💰 **Price:** {property_data['asking_price']:}, ETB\n\n"
    )

    try:
        await app.bot.edit_message_text(
            chat_id=TARGET_CHANNEL_ID,
            message_id=message_id,
            text=updated_text,
            parse_mode='Markdown'
        )
    except Exception as e:
        print(f"Error updating message: {e}")

```

```
# Callback for Privacy Phone Number Reveal
async def handle_contact_click(update: Update, context: ContextTypes.DEFAULT_TYPE):
    query = update.callback_query
    await query.answer()
    phone = query.data.split("_")[1]
    await query.message.reply_text(f" Registrant Contact: {phone}", ephemeral=True)
```

3. Mobile Registration Form (React + Telegram WebApp SDK)

This component lets agents submit property info, upload multiple photos, and set custom commission rates.

```
import React, { useState } from 'react';
import { retrieveLaunchParams } from '@telegram-apps/sdk';
import { supabase } from '../services/supabaseClient';

export const PropertyForm: React.FC = () => {
    const [formData, setFormData] = useState({
        category: 'sale',
        site_name: 'Ayat 49',
        block_no: '',
        floor_no: '',
        house_no: '',
        bedrooms: '2-bed',
        bathrooms: 1,
        asking_price: '',
        commission_percent: '2.00',
        phone_number: '',
    });

    const [files, setFiles] = useState<FileList | null>(null);
    const [loading, setLoading] = useState(false);

    const handleSubmit = async (e: React.FormEvent) => {
        e.preventDefault();
        setLoading(true);

        try {
            // 1. Upload Images to Supabase Storage
            const uploadedUrls: string[] = [];
            if (files) {
                for (let i = 0; i < files.length; i++) {
                    const file = files[i];
                    const fileName = `${Date.now()}_${file.name}`;
                    const { data, error } = await supabase.storage
                        .from('condo-images')
                        .upload(fileName, file);

                    if (data) {
                        const publicUrl = supabase.storage.from('condo-images').getPublicUrl(fileName).data.publicUrl;
                        uploadedUrls.push(publicUrl);
                    }
                }
            }

            // 2. Insert Listing into Postgres
            const { data: newProperty, error } = await supabase
                .from('properties')
                .insert([
                    {
                        ...formData,
                        asking_price: parseFloat(formData.asking_price),
                        commission_percent: parseFloat(formData.commission_percent),
                        image_urls: uploadedUrls,
                    }
                ])
                .select()
                .single();

            if (error) throw error;

            alert('Property Registered & Broadcasted Successfully!');
        } catch (err: any) {
            alert(`Error: ${err.message}`);
        } finally {
            setLoading(false);
        }
    };

    return (
        <form onSubmit={handleSubmit} style={{ padding: '16px', display: 'flex', flexDirection: 'column', gap: '12px' }}>
            <h2>Register Condo Property</h2>

            <label>Category</label>
            <select value={formData.category} onChange={e => setFormData({...formData, category: e.target.value}}>
                <option value="sale">For Sale</option>
                <option value="rental">For Rent</option>
                <option value="guesthouse">Guest House</option>
            </select>

            <label>Condo Site Name</label>
            <select value={formData.site_name} onChange={e => setFormData({...formData, site_name: e.target.value}}>
                <option value="Ayat 49">Ayat 49</option>
                <option value="Gotera">Gotera</option>
                <option value="Summit">Summit</option>
                <option value="Koye Feche">Koye Feche</option>
                <option value="Jeka">Jeka</option>
                <option value="Bole Bulbula">Bole Bulbula</option>
            </select>

            <div style={{ display: 'flex', gap: '8px' }}>
```

```

        <input placeholder="Block No" required onChange={e => setFormData({...formData, block_no: e.target.value})} />
        <input placeholder="Floor No" required onChange={e => setFormData({...formData, floor_no: e.target.value})} />
        <input placeholder="House No" required onChange={e => setFormData({...formData, house_no: e.target.value})} />
    </div>

    <div style={{ display: 'flex', gap: '8px' }}>
        <select value={formData.bedrooms} onChange={e => setFormData({...formData, bedrooms: e.target.value})}>
            <option value="studio">Studio</option>
            <option value="1-bed">1 Bed</option>
            <option value="2-bed">2 Bed</option>
            <option value="3-bed">3 Bed</option>
        </select>
        <input type="number" placeholder="Price (ETB)" required onChange={e => setFormData({...formData, asking_price: e.target.value})} />
    </div>

    <label>Commission Rate (%)</label>
    <input type="number" step="0.1" value={formData.commission_percent} onChange={e => setFormData({...formData, commission_percent: e.target.value})} />

    <label>Contact Phone Number</label>
    <input type="tel" placeholder="+251..." required onChange={e => setFormData({...formData, phone_number: e.target.value})} />

    <label>Upload Photos</label>
    <input type="file" multiple accept="image/*" onChange={e => setFiles(e.target.files)} />

    <button type="submit" disabled={loading} style={{ padding: '12px', background: '#0088cc', color: '#fff', border: 'none', borderRadius: '8px' }}>
        {loading ? 'Registering...' : 'Register & Post Listing'}
    </button>
</form>
);
};

```

User prompt: yes

Response: Here is the complete implementation for the Supabase Database Webhook integration and the Agent Listing Management React UI.

1. Supabase Webhooks & Python Listener Configuration

When a new listing is inserted or updated in Supabase, Supabase automatically emits a POST payload containing the row data.

A. SQL Trigger Setup

Run this snippet in the Supabase SQL Editor to send webhooks whenever a listing is created or its status changes:

```

-- Trigger to invoke HTTP request on property changes
create trigger "on_property_change"
after insert or update of status
on "public"."properties"
for each row
execute function "supabase_functions"."http_request"(
    'https://your-bot-server.com/webhook/supabase', -- Your Python server URL
    'POST',
    '{"Content-Type":"application/json"}',
    '{}',
    '5000'
);

```

B. Python FastAPI Webhook Endpoint (webhook_listener.py)

This lightweight server receives the Supabase webhook event and instructs the Telegram Bot to post a new broadcast or edit an existing message:

```

from fastapi import FastAPI, Request
import httpx
import os

app = FastAPI()

BOT_TOKEN = os.getenv("TELEGRAM_BOT_TOKEN")
CHANNEL_ID = "@your_condo_channel_id"

@app.post("/webhook/supabase")
async def handle_supabase_event(request: Request):
    payload = await request.json()
    event_type = payload.get("type") # 'INSERT' or 'UPDATE'
    record = payload.get("record")
    old_record = payload.get("old_record")

    async with httpx.AsyncClient() as client:
        # 1. HANDLE NEW PROPERTY INSERTION
        if event_type == "INSERT":
            text = (
                f"🟢 [AVAILABLE]\n"
                f"🏠 **CONDO FOR {record['category'].upper()}**\n\n"
                f"📍 **Location:** {record['site_name']} (Block {record['block_no']}, Floor {record['floor_no']})\n\n"
                f"🛏️ **Bedrooms:** {record['bedrooms']}\n\n"
                f"💰 **Price:** {record['asking_price']:,} ETB\n\n"
                f"💵 **Commission Rate:** {record['commission_percent']}%\n\n"
            )
            keyboard = {
                "inline_keyboard": [[
                    {"text": "📞 Contact Agent / Owner", "callback_data": f"contact_{record['user_id']}"}
                ]]
            }

            # Send to Telegram Channel
            resp = await client.post(

```

```

    f"https://api.telegram.org/bot{BOT_TOKEN}/sendMessage",
    json={
      "chat_id": CHANNEL_ID,
      "text": text,
      "parse_mode": "Markdown",
      "reply_markup": keyboard
    }
  )
  msg_data = resp.json()
  telegram_msg_id = msg_data["result"]["message_id"]

  # Save the telegram message_id back to Supabase for future updates
  # (Requires setting telegram_message_id = telegram_msg_id in public.properties)

  # 2. HANDLE STATUS UPDATE (Sold / Rented / Occupied)
  elif event_type == "UPDATE" and record.get("status") != old_record.get("status"):
    msg_id = record.get("telegram_message_id")
    if msg_id:
      status_emoji = "🟢" if record["status"] == "available" else "🔴"
      updated_text = (
        f"{status_emoji} **[STATUS: {record['status'].upper()}]**\n"
        f"🏠 **CONDO FOR {record['category'].upper()}**\n\n"
        f"📍 **Location:** {record['site_name']} (Block {record['block_no']})\n"
        f"🛏️ **Bedrooms:** {record['bedrooms']}\n"
        f"💰 **Price:** {record['asking_price']}, ETB\n"
      )
      await client.post(
        f"https://api.telegram.org/bot{BOT_TOKEN}/editMessageText",
        json={
          "chat_id": CHANNEL_ID,
          "message_id": msg_id,
          "text": updated_text,
          "parse_mode": "Markdown"
        }
      )

    return {"status": "success"}

```

2. Agent Management Dashboard UI (AgentListings.tsx)

This React component gives agents a clean interface inside the Telegram Mini App to view their registered properties and instantly update property statuses (Available, Sold, Rented, Occupied).

```

import React, { useEffect, useState } from 'react';
import { supabase } from '../services/supabaseClient';

interface Property {
  id: string;
  site_name: string;
  block_no: string;
  house_no: string;
  category: string;
  asking_price: number;
  status: 'available' | 'sold' | 'rented' | 'occupied';
  image_urls: string[];
}

export const AgentListings: React.FC<{ userId: string }> = ({ userId }) => {
  const [properties, setProperties] = useState<Property[]>([]);
  const [loading, setLoading] = useState(true);

  // Fetch agent's registered properties
  const fetchMyProperties = async () => {
    setLoading(true);
    const { data, error } = await supabase
      .from('properties')
      .select('*')
      .eq('user_id', userId)
      .order('created_at', { ascending: false });

    if (data) setProperties(data);
    setLoading(false);
  };

  useEffect(() => {
    fetchMyProperties();
  }, [userId]);

  // Update Status in Supabase (Triggers automatic Telegram broadcast update)
  const handleStatusChange = async (propertyId: string, newStatus: string) => {
    const { error } = await supabase
      .from('properties')
      .update({ status: newStatus })
      .eq('id', propertyId);

    if (!error) {
      setProperties(prev =>
        prev.map(p => (p.id === propertyId ? { ...p, status: newStatus as any } : p))
      );
    } else {
      alert(`Failed to update status: ${error.message}`);
    }
  };

  if (loading) return <div style={{ padding: '16px' }}>Loading your listings...</div>;

```

```
return (
  <div style={{ padding: '16px', display: 'flex', flexDirection: 'column', gap: '16px' }}>
    <h3>My Registered Properties</h3>

    {properties.length === 0 ? (
      <p>No properties registered yet.</p>
    ) : (
      properties.map(p => (
        <div
          key={p.id}
          style={{
            background: 'var(--tg-theme-secondary-bg-color, #f5f5f5)',
            padding: '12px',
            borderRadius: '10px',
            borderLeft: p.status === 'available' ? '4px solid #4CAF50' : '4px solid #F44336'
          }}
        >
          <div style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'center' }}>
            <strong>{p.site_name} - Blk {p.block_no}, House {p.house_no}</strong>
            <small style={{ textTransform: 'uppercase', fontWeight: 'bold' }}>{p.category}</small>
          </div>

          <p style={{ margin: '8px 0', fontSize: '14px' }}>
            Price: <strong>{p.asking_price.toLocaleString()} ETB</strong>
          </p>

          <div style={{ marginTop: '10px', display: 'flex', alignItems: 'center', gap: '8px' }}>
            <label style={{ fontSize: '12px' }}>Update Status:</label>
            <select
              value={p.status}
              onChange={e => handleStatusChange(p.id, e.target.value)}
              style={{
                padding: '6px',
                borderRadius: '6px',
                border: '1px solid #ccc',
                background: 'var(--tg-theme-bg-color, #fff)'
              }}
            >
              <option value="available">🟢 Available</option>
              <option value="sold">🔴 Sold</option>
              <option value="rented">🟡 Rented</option>
              <option value="occupied">🔴 Occupied</option>
            </select>
          </div>
        </div>
      ))
    )}
  </div>
);
```

3. End-to-End Testing Procedure

- 1

Run Local Tunnel for Webhook Testing

Ngrok or Cloudflare Tunnel

Start your Python FastAPI server locally on port 8000. Expose it using Ngrok (ngrok http 8000) and paste the HTTPS tunnel URL into your Supabase HTTP trigger SQL function.
- 2

Test Property Submission

Form Verification

Open the React WebApp via Telegram Bot, complete the condo registration form (Block, Floor, House, Price, Commission, Phone, Photos), and hit submit. Verify the new row appears in your Supabase properties table.
- 3

Verify Automated Channel Broadcast

Telegram Broadcast Sync

Check your target Telegram channel. A new formatted post should appear instantly featuring the condo details and the inline button hiding the owner's phone number.
- 4

Test Listing Status Edit

Real-time Status Edit

In the Mini App, navigate to **My Listings**, select a property, and change the status from Available to Rented or Sold. Verify the original channel post automatically updates its header text to 🟡 [STATUS: RENTED].

Sources

1. [Database Webhooks | Supabase Docs](#)

User prompt: list the steps for a beginner to deploy and test the app

Response: Deploying and testing your Telegram Condo Listing app involves three main layers: setting up the database/storage (Supabase), hosting the frontend (Vite/React), and running the webhook/bot server (FastAPI + Python Telegram Bot).
Here is the step-by-step beginner guide to getting everything live and tested.

Phase 1: Database & File Storage Setup (Supabase)

1. Create a Supabase Project:
- Go to [Supabase](#), create an account, and start a new project.

Save your **Project URL** and anon **Public Key** from Project Settings > API.

2. Set Up the Database Tables:

- Open the **SQL Editor** in your Supabase dashboard.
- Paste and run the schema SQL script (creates the users and properties tables with their respective enums).

3. Create Storage Bucket for Photos:

- Go to **Storage** in the left sidebar and click **New Bucket**.
- Name it `condo-images`.
- Toggle **Public Bucket** to ON so that Telegram can fetch and display images posted by agents.

Phase 2: Frontend Deployment (Vite + React)

1. Push Code to GitHub:

- Initialize a local Git repository in your React project folder, commit your changes, and push the repository to GitHub.

2. Deploy Frontend to Vercel or Netlify:

- Sign up at [Vercel](#) and click **Add New > Project**.
- Import your GitHub repository.
- Add environment variables for `VITE_SUPABASE_URL` and `VITE_SUPABASE_ANON_KEY`.
- Click **Deploy**. Vercel will generate an SSL-enabled HTTPS URL (e.g., `https://your-app.vercel.app`).

Phase 3: Telegram Bot & Channel Setup

1. Create Bot via BotFather:

- Open Telegram and search for **@BotFather**.
- Send `/newbot`, follow the prompts, and copy the **HTTP API Token**.
- Send `/setmenubutton`, select your bot, enter a button title (e.g., *"Open Condo App"*), and paste your hosted Vercel URL (`https://your-app.vercel.app`).

2. Configure Your Telegram Channel:

- Create a public or private Telegram Channel or Group where listings will be broadcasted.
- Add your newly created Bot as an **Administrator** of that Channel with permission to *Post Messages* and *Edit Messages*.
- Note down your channel username (e.g., `@your_condo_channel1`) or channel ID.

Phase 4: Webhook & Bot Backend Deployment

1. Deploy FastAPI Backend to Render:

- Sign up at [Render](#) and select **New > Web Service**.
- Connect your backend code repository (containing `webhook_listener.py` and `requirements.txt`).
- Set Environment Variables:
 - `TELEGRAM_BOT_TOKEN` = *[Your Bot Token]*
 - `TARGET_CHANNEL_ID` = *[Your Channel Username/ID]*
- Click **Deploy**. Render will provide a live public HTTPS URL (e.g., `[https://your-backend.onrender.com](https://your-backend.onrender.com)`).

2. Connect Supabase Webhook to Render Backend:

- Go to your **Supabase Dashboard > Database > Webhooks**.
- Click **Create a Webhook**.
- Name: `property_change_webhook`.
- Table: `properties`.
- Events: Check Insert and Update.
- Webhook URL: `[https://your-backend.onrender.com/webhook/supabase](https://your-backend.onrender.com/webhook/supabase)`.
- Save the webhook.

Phase 5: Step-by-Step Testing Protocol

1 1. Launch the Mini App

Open Telegram App

Open Telegram on your mobile phone or desktop, search for your bot, and click the menu button (*"Open Condo App"*) to verify the React UI loads inside the Telegram overlay.

2 2. Submit a Test Listing

Form & Media Check

Fill out the property registration form with test data (Site: *Ayat 49*, Block, Floor, House, Price, Phone Number), attach 1–2 test photos, and tap **Register & Post Listing**.

3 3. Verify Storage & Database Insertion

Database Audit

Check your Supabase dashboard:

- In **Table Editor**, confirm a new row exists in `properties`.
- In **Storage (condo-images)**, confirm the uploaded photos are visible.

4 4. Verify Channel Broadcast

Broadcast Sync

Open your Telegram Channel. Confirm that the bot automatically posted the listing formatted with location, pricing, commission, media, and the "📞 *Contact Agent / Owner*" button.

5 5. Test Status Update (Sold/Rented)

Real-Time Edit Sync

In your Mini App, navigate to **My Listings**, select the test property, and change its status from Available to Rented or Sold. Recheck your Telegram Channel post to verify the header automatically updated to  [STATUS: RENTED].
